

Table of contents

1. Preface	1
Preface	3
1.1. Who this book is for	3
1.2. What this book covers	3
1.3. What this book does not cover	3
1.4. How to run the code	4
1.5. Conventions	4
1.6. Acknowledgments	4
2. Introduction: Why Strategy in R?	5
3. Introduction	7
3.1. The computational turn in game theory	7
3.2. Why R?	7
3.3. The structure of this book	7
3.4. How to use this book	8
I. Part I — Foundations of Game Theory	9
4. What is a Game?	11
5. What is a Game?	13
Learning objectives	13
5.1. Motivation	13
5.2. Theory	14
5.2.1. The four building blocks	14
5.2.2. What makes a situation a “game”?	14
5.2.3. Three canonical examples	14
5.3. Implementation in R	15
5.4. Worked example	17
5.5. Extensions	18
Exercises	18
6. Normal-Form Games	21
7. Normal-Form Games	23
Learning objectives	23
7.1. Motivation	23
7.2. Theory	23
7.2.1. The normal-form representation	23
7.2.2. Dominance	24
7.2.3. Iterated elimination of strictly dominated strategies (IESDS)	24
7.2.4. Classic 2x2 games	24

7.3. Implementation in R	25
7.3.1. Checking for strict dominance	25
7.3.2. Publication figure: classic 2x2 payoff structures	26
7.4. Worked example	27
7.4.1. Displaying the full payoff matrix with <code>gt</code>	31
7.5. Extensions	32
Exercises	33
8. Nash Equilibrium	35
9. Nash Equilibrium	37
Learning objectives	37
9.1. Motivation	37
9.2. Theory	37
9.2.1. Best responses	37
9.2.2. Nash equilibrium defined	37
9.2.3. Existence	38
9.2.4. Multiplicity	38
9.3. Implementation in R	38
9.3.1. Finding pure-strategy Nash equilibria	38
9.3.2. Battle of the Sexes — multiple equilibria	39
9.3.3. Best-response correspondence plot	40
9.4. Worked example	42
9.5. Extensions	43
Exercises	44
10. Mixed Strategies and the Indifference Principle	45
11. Mixed Strategies and the Indifference Principle	47
Learning objectives	47
11.1. Motivation	47
11.2. Theory	47
11.2.1. Mixed strategies defined	47
11.2.2. Expected payoffs under mixing	48
11.2.3. The indifference principle	48
11.2.4. Computing mixed NE in 2x2 games	48
11.2.5. Nash's existence theorem	49
11.3. Implementation in R	49
11.3.1. Computing mixed equilibria with <code>solve_2x2_mixed_nash()</code>	49
11.3.2. Visualizing the indifference point	50
11.4. Worked example	52
11.4.1. Matching Pennies: step-by-step	52
11.5. Extensions	53
Exercises	54
12. Extensive-Form Games	55
13. Extensive-Form Games	57
Learning objectives	57
13.1. Motivation	57
13.2. Theory	57
13.2.1. Game trees	57
13.2.2. Strategies in extensive form vs. normal form	58

13.2.3. Backward induction	58
13.2.4. Subgame perfect equilibrium	58
13.3. Implementation in R	59
13.3.1. Defining the game tree	59
13.3.2. Drawing the game tree	59
13.3.3. Backward induction algorithm	61
13.4. Worked example	63
13.5. Extensions	65
Exercises	65
14. Bayesian Games	67
15. Bayesian Games	69
Learning objectives	69
15.1. Motivation	69
15.2. Theory	69
15.2.1. The Bayesian game model	69
15.2.2. Bayesian Nash equilibrium	70
15.2.3. Signaling games	70
15.2.4. Separating vs. pooling equilibria	70
15.3. Implementation in R	71
15.3.1. Expected payoffs under each equilibrium type	71
15.3.2. Checking incentive compatibility	73
15.3.3. Publication figure: expected payoffs by type and equilibrium	73
15.4. Worked example	74
15.5. Extensions	75
Exercises	76
16. Repeated Games	77
17. Repeated Games	79
Learning objectives	79
17.1. Motivation	79
17.2. Theory	79
17.2.1. The stage game	79
17.2.2. Finite vs. infinite horizon	80
17.2.3. Trigger strategies	80
17.2.4. The Grim Trigger condition	80
17.2.5. The folk theorem	81
17.3. Implementation in R	81
17.3.1. Stage game setup and Grim Trigger analysis	81
17.3.2. Payoff comparison across discount factors	81
17.3.3. Folk theorem payoff region	82
17.3.4. Publication figure: folk theorem payoff region	83
17.4. Worked example	85
17.4.1. Comparing Grim Trigger and Tit-for-Tat	87
17.5. Extensions	87
Exercises	87
18. Cooperative Game Theory	89
19. Cooperative Game Theory	91
Learning objectives	91

19.1. Motivation	91
19.2. Theory	91
19.2.1. Characteristic function games	91
19.2.2. Imputations	92
19.2.3. The core	92
19.2.4. The Shapley value	92
19.3. Implementation in R	93
19.3.1. Defining a cooperative game	93
19.3.2. Computing the Shapley value	94
19.3.3. Checking the core	95
19.3.4. Publication figure: Shapley value comparison	96
19.3.5. Marginal contributions table	97
19.4. Worked example	98
19.5. Extensions	100
Exercises	100
II. Part II — The R Toolkit	103
20. The R Environment for Strategic Computation	105
21. The R Environment	107
Learning objectives	107
21.1. Motivation	107
21.2. Theory	108
21.2.1. The reproducibility stack	108
21.2.2. Why IDE configuration matters for game theory	108
21.2.3. The shared setup pattern	108
21.3. Implementation in R	109
21.3.1. Project directory layout	109
21.3.2. Benchmarking vectorized vs. loop-based payoff computation	110
21.3.3. The <code>_common.R</code> file explained	112
21.3.4. Initializing <code>renv</code>	113
21.4. Worked example	113
21.5. Extensions	115
Exercises	116
22. The GameTheory Package	117
23. The GameTheory Package	119
Learning objectives	119
23.1. Motivation	119
23.2. Theory	119
23.2.1. Cooperative games and characteristic functions	119
23.2.2. The Shapley value	120
23.2.3. The Banzhaf power index	120
23.2.4. The GameTheory package API	120
23.3. Implementation in R	121
23.3.1. Defining a characteristic function	121
23.3.2. Computing the Shapley value via permutations	121
23.3.3. Shapley value via the analytical formula	123
23.3.4. Computing the Banzhaf power index	123

23.3.5. Comparing power indices across coalition structures	124
23.4. Worked example	126
23.4.1. A simplified EU Council vote	126
23.5. Extensions	128
Exercises	129
24. The CoopGame Package	131
25. The CoopGame Package	133
Learning objectives	133
25.1. Motivation	133
25.2. Theory	133
25.2.1. Characteristic function representation	133
25.2.2. The core	134
25.2.3. The nucleolus	134
25.2.4. The CoopGame package API	134
25.3. Implementation in R	134
25.3.1. Coalition encoding and game definition	134
25.3.2. Computing the core via linear constraints	135
25.3.3. Finding core vertices	136
25.3.4. Computing the nucleolus	137
25.3.5. Shapley value and core visualization	138
25.4. Worked example	140
25.4.1. Step-by-step core membership check	140
25.5. Extensions	142
Exercises	142
26. The gtree Package	143
27. The gtree Package	145
Learning objectives	145
27.1. Motivation	145
27.2. Theory	145
27.2.1. The gtree DSL	145
27.2.2. Gambit integration	146
27.2.3. Backward induction for perfect-information games	147
27.2.4. The ultimatum game	147
27.3. Implementation in R	147
27.3.1. A flexible game tree representation	147
27.3.2. Building the ultimatum game tree	148
27.3.3. Visualizing the game tree	149
27.4. Worked example	152
27.4.1. Scaling analysis	154
27.5. Extensions	156
Exercises	157
28. nashpy via reticulate	159
29. nashpy via reticulate	161
Learning objectives	161
29.1. Motivation	161
29.2. Theory	161
29.2.1. The reticulate bridge	161

29.2.2. Support enumeration	162
29.2.3. Complexity considerations	163
29.3. Implementation in R	163
29.3.1. General support enumeration solver	163
29.3.2. Testing on the three canonical 2x2 games	165
29.3.3. Cross-validation with the 2x2 solver	167
29.4. Worked example	168
29.4.1. The 2x2 Hawk-Dove game	168
29.4.2. A 3x3 extension: Rock-Paper-Scissors	169
29.4.3. Comparative visualization	170
29.5. Extensions	172
Exercises	173
30. Building Custom Solvers in R	175
31. Building Custom Solvers in R	177
Learning objectives	177
31.1. Motivation	177
31.2. Theory	177
31.2.1. Support enumeration (recap)	177
31.2.2. The Lemke–Howson algorithm	178
31.2.3. Fictitious play	178
31.3. Implementation in R	179
31.3.1. Lemke–Howson sketch for 2x2 games	179
31.3.2. Fictitious play implementation	180
31.3.3. Testing on Matching Pennies	182
31.4. Worked example	182
31.4.1. Setting up the game	183
31.4.2. Running fictitious play	183
31.4.3. Publication figure: convergence trajectory	184
31.4.4. Comparing all three approaches	186
31.4.5. Convergence speed across games	188
31.5. Extensions	189
Exercises	190
III. Part III — Simulation	191
32. Monte Carlo Methods for Games	193
33. Monte Carlo Methods for Games	195
Learning objectives	195
33.1. Motivation	195
33.2. Theory	195
33.2.1. Monte Carlo estimation	195
33.2.2. Convergence rate and confidence intervals	196
33.2.3. Bootstrap confidence intervals	196
33.3. Implementation in R	196
33.3.1. Mixed-strategy payoff estimation	196
33.3.2. Convergence plot	197
33.3.3. Payoff distribution under mixed strategies	198

33.4. Worked example	200
33.4.1. Bootstrap confidence interval	201
33.4.2. Verifying convergence	202
33.5. Extensions	203
Exercises	203
34. Agent-Based Models for Strategic Interaction	205
35. Agent-Based Models for Strategic Interaction	207
Learning objectives	207
35.1. Motivation	207
35.2. Theory	207
35.2.1. The Hawk-Dove game	207
35.2.2. Agent-based evolutionary dynamics	208
35.2.3. Connection to replicator dynamics	208
35.3. Implementation in R	209
35.3.1. Core simulation functions	209
35.3.2. Running the ABM	210
35.3.3. Strategy frequency evolution	210
35.3.4. Population fitness over time	212
35.4. Worked example	213
35.4.1. Sensitivity to parameters	215
35.5. Extensions	216
Exercises	216
36. Replicating Axelrod's Tournament	217
37. Replicating Axelrod's Tournament	219
Learning objectives	219
37.1. Motivation	219
37.2. Theory	219
37.2.1. The iterated Prisoner's Dilemma	219
37.2.2. Axelrod's four properties of successful strategies	220
37.3. Implementation in R	220
37.3.1. Strategy definitions	220
37.3.2. Round-robin tournament	221
37.3.3. Visualizing the results	222
37.3.4. Head-to-head heatmap	223
37.4. Worked example	223
37.5. Extensions	224
Exercises	225
38. The Spatial Prisoner's Dilemma	227
39. Spatial Prisoner's Dilemma	229
Learning objectives	229
39.1. Motivation	229
39.2. Theory	229
39.2.1. From well-mixed to spatial populations	229
39.2.2. The Nowak-May model	230
39.2.3. Why clusters protect cooperators	230
39.3. Implementation in R	230
39.3.1. Grid initialization and neighbor scoring	230

39.3.2. Update rule: imitate the best neighbor	231
39.3.3. Running the simulation	231
39.3.4. Visualizing a grid snapshot	232
39.4. Worked example	232
39.5. Extensions	235
Exercises	235
40. Replicator Dynamics	237
41. Replicator Dynamics	239
Learning objectives	239
41.1. Motivation	239
41.2. Theory	239
41.2.1. The replicator equation	239
41.2.2. Key properties	240
41.2.3. Two example games	240
41.3. Implementation in R	240
41.3.1. Solving with <code>run_replicator()</code>	240
41.3.2. Simplex coordinate transform	241
41.3.3. Running RPS dynamics from multiple initial conditions	241
41.3.4. Running Hawk-Dove dynamics	241
41.4. Worked example	242
41.4.1. Rock-Paper-Scissors	242
41.4.2. Hawk-Dove	244
41.4.3. Comparing the two games	246
41.5. Extensions	246
Exercises	246
42. Evolutionarily Stable Strategies	249
43. Evolutionarily Stable Strategies	251
Learning objectives	251
43.1. Motivation	251
43.2. Theory	251
43.2.1. The ESS definition	251
43.2.2. Relationship to Nash equilibrium	252
43.2.3. Invasion dynamics	252
43.3. Implementation in R	252
43.3.1. Checking ESS conditions for a symmetric game	252
43.3.2. Testing on the Hawk-Dove game	253
43.3.3. Invasion fitness landscape	254
43.3.4. Basins of attraction in a 3-strategy game	256
43.4. Worked example	258
43.4.1. Step 1: Check pure strategies	258
43.4.2. Step 2: Interpret the results	259
43.4.3. Step 3: Verify with replicator dynamics	261
43.5. Extensions	261
Exercises	262
44. Games on Networks	263
45. Games on Networks	265
Learning objectives	265

45.1. Motivation	265
45.2. Theory	265
45.2.1. Networks as adjacency matrices	265
45.2.2. Games on networks	265
45.2.3. Small-world networks	266
45.3. Implementation in R	266
45.3.1. Network generators	266
45.3.2. Network statistics	267
45.3.3. Circular layout and network visualization	268
45.3.4. Prisoner's Dilemma on networks	269
45.3.5. Comparing cooperation across topologies	270
45.4. Worked example	272
45.5. Extensions	273
Exercises	273
46. Performance Optimization	275
47. Performance Optimization	277
Learning objectives	277
47.1. Motivation	277
47.2. Theory	277
47.2.1. Why R loops are slow	277
47.2.2. Three levels of optimization	277
47.2.3. The profiling workflow	278
47.3. Implementation in R	278
47.3.1. The baseline: loop-based spatial PD	278
47.3.2. The optimized version: vectorized spatial PD	279
47.3.3. Benchmarking the two implementations	280
47.3.4. Verifying correctness	281
47.3.5. Visualizing the benchmark	281
47.3.6. Scaling behavior	282
47.4. Worked example	284
47.4.1. Step 1: Profile the baseline	284
47.4.2. Step 2: Vectorize payoff computation	285
47.4.3. Step 3: Memory pre-allocation matters	285
47.4.4. When to consider Rcpp	286
47.5. Extensions	287
Exercises	287
IV. Part IV — AI and Machine Learning	289
48. Machine Learning Foundations for Game Theory	291
49. ML Foundations for Game Theory	293
Learning objectives	293
49.1. Motivation	293
49.2. Theory	293
49.2.1. Supervised learning as a game	293
49.2.2. Online learning and regret	294
49.2.3. Regret matching	294
49.2.4. Feature representation for game-theoretic data	294

49.3. Implementation in R	294
49.3.1. Regret matching for Rock-Paper-Scissors	294
49.3.2. Decision boundary visualization	297
49.4. Worked example	298
49.4.1. Predicting cooperation in the iterated Prisoner's Dilemma	298
49.5. Extensions	301
Exercises	301
50. Reinforcement Learning Foundations	303
51. Reinforcement Learning Foundations	305
Learning objectives	305
51.1. Motivation	305
51.2. Theory	305
51.2.1. Markov decision processes	305
51.2.2. The Bellman equation	306
51.2.3. Tabular Q-learning	306
51.2.4. SARSA	306
51.2.5. Exploration vs exploitation	306
51.3. Implementation in R	306
51.3.1. Gridworld environment	306
51.3.2. Tabular Q-learning agent	307
51.3.3. SARSA agent	308
51.3.4. Training on the gridworld	309
51.3.5. Exploration comparison: epsilon-greedy vs UCB	311
51.4. Worked example	312
51.5. Extensions	313
Exercises	313
52. Q-Learning in 2-Player Games	315
53. Q-Learning in Games	317
Learning objectives	317
53.1. Motivation	317
53.2. Theory	317
53.2.1. Q-learning recap	317
53.2.2. Independent learners	318
53.2.3. Convergence properties	318
53.3. Implementation in R	318
53.3.1. Q-learning agent	318
53.3.2. Running a repeated game	319
53.3.3. Coordination game: convergence	319
53.3.4. Zero-sum game: cycling	320
53.4. Worked example	322
53.5. Extensions	323
Exercises	323
54. Multi-Agent Reinforcement Learning	325
55. Multi-Agent RL	327
Learning objectives	327
55.1. Motivation	327

55.2. Theory	327
55.2.1. Independent vs. joint-action learners	327
55.2.2. Minimax-Q learning	328
55.2.3. MADDPG: centralized training, decentralized execution	328
55.2.4. Policy Space Response Oracles (PSRO)	329
55.3. Implementation in R	329
55.3.1. Joint-action learner	329
55.3.2. Minimax-Q solver	330
55.3.3. Minimax-Q agent	330
55.3.4. Simulation: independent vs. joint-action learners	330
55.3.5. Win-rate matrix across strategies	332
55.4. Worked example	334
55.5. Extensions	336
Exercises	336
56. Self-Play and AlphaZero	339
57. Self-Play and AlphaZero	341
Learning objectives	341
57.1. Motivation	341
57.2. Theory	341
57.2.1. Self-play as a training methodology	341
57.2.2. Monte Carlo Tree Search	342
57.2.3. AlphaZero architecture	342
57.3. Implementation in R	342
57.3.1. Tic-Tac-Toe game engine	342
57.3.2. MCTS implementation	343
57.3.3. Running MCTS	344
57.3.4. MCTS tree visualization	345
57.3.5. Self-play win-rate tracking	347
57.4. Worked example	348
57.5. Extensions	350
Exercises	350
58. Counterfactual Regret Minimization and Poker	353
59. CFR and Poker	355
Learning objectives	355
59.1. Motivation	355
59.2. Theory	355
59.2.1. Imperfect information and information sets	355
59.2.2. Regret matching	355
59.2.3. CFR algorithm	356
59.2.4. Kuhn Poker	356
59.3. Implementation in R	356
59.3.1. Regret matching	356
59.3.2. Kuhn Poker CFR	357
59.3.3. Run CFR	359
59.3.4. Figure 1: Strategy convergence	359
59.3.5. Figure 2: Exploitability	360
59.4. Worked example	361
59.5. Extensions	363

Exercises	363
60. GANs as Minimax Games	365
61. GANs as Minimax Games	367
Learning objectives	367
61.1. Motivation	367
61.2. Theory	367
61.2.1. The minimax formulation	367
61.2.2. Nash equilibrium of the GAN game	368
61.2.3. Optimal discriminator	368
61.2.4. Mode collapse	368
61.3. Implementation in R	368
61.3.1. Target and generator distributions	368
61.3.2. Discriminator on a grid	369
61.3.3. Training loop	369
61.3.4. Figure 1: Generator distribution evolution	371
61.3.5. Figure 2: Discriminator accuracy	372
61.4. Worked example	373
61.5. Extensions	375
Exercises	375
62. LLM Agents and Strategic Reasoning	377
63. LLM Agents and Strategy	379
Learning objectives	379
63.1. Motivation	379
63.2. Theory	379
63.2.1. LLMs as game-playing agents	379
63.2.2. Level-k reasoning	380
63.2.3. Cognitive hierarchy	380
63.2.4. The p-beauty contest	380
63.2.5. Bounded rationality and LLMs	380
63.3. Implementation in R	381
63.3.1. Level-k beauty contest simulation	381
63.3.2. Run the simulation	381
63.3.3. Figure 1: Level-k action distributions	382
63.3.4. Figure 2: Strategy performance comparison	383
63.4. Worked example	385
63.5. Extensions	388
Exercises	388
V. Part V — Applications	389
64. Auction Theory	391
65. Auction Theory	393
Learning objectives	393
65.1. Motivation	393
65.2. Theory	393
65.2.1. Auction formats	393
65.2.2. Dominant strategy in second-price auctions	394

65.2.3. Optimal bidding in first-price auctions	394
65.2.4. Winner's expected payment	394
65.2.5. Revenue equivalence theorem	394
65.3. Implementation in R	395
65.3.1. Auction simulation functions	395
65.3.2. Figure 1: Revenue comparison across auction types	395
65.3.3. Figure 2: Optimal bid function in first-price auction	396
65.4. Worked example	397
65.5. Extensions	399
Exercises	400
66. Mechanism Design	401
67. Mechanism Design	403
Learning objectives	403
67.1. Motivation	403
67.2. Theory	403
67.2.1. The mechanism design problem	403
67.2.2. The revelation principle	403
67.2.3. Incentive compatibility and individual rationality	404
67.2.4. The VCG mechanism	404
67.3. Implementation in R	404
67.3.1. VCG mechanism for public goods	404
67.3.2. Comparing mechanisms across many instances	405
67.3.3. Figure 1: Social welfare comparison	406
67.3.4. Figure 2: VCG transfer payments by agent	407
67.4. Worked example	408
67.5. Extensions	411
Exercises	411
68. Matching Markets	413
69. Matching Markets	415
Learning objectives	415
69.1. Motivation	415
69.2. Theory	415
69.2.1. Stable matching	415
69.2.2. The Gale-Shapley algorithm	416
69.2.3. Strategy-proofness	416
69.3. Implementation in R	416
69.3.1. Gale-Shapley algorithm	416
69.3.2. Helper: compute welfare	417
69.3.3. Figure 1: Matching visualization	418
69.3.4. Figure 2: Proposer advantage	419
69.4. Worked example	421
69.5. Extensions	424
Exercises	424
70. Bargaining Theory	425
71. Bargaining Theory	427
Learning objectives	427
71.1. Motivation	427

71.2. Theory	427
71.2.1. The bargaining problem	427
71.2.2. Nash bargaining solution	427
71.2.3. Asymmetric Nash bargaining	428
71.2.4. Rubinstein alternating-offers model	428
71.3. Implementation in R	429
71.3.1. Nash bargaining solution computation	429
71.3.2. Nash bargaining visualization	429
71.3.3. Rubinstein alternating-offers model	431
71.3.4. Rubinstein equilibrium shares by discount factor	432
71.4. Worked example	433
71.4.1. Labor-management negotiation	433
71.5. Extensions	435
Exercises	435
72. Conflict and Cooperation Models	437
73. Conflict and Cooperation Models	439
Learning objectives	439
73.1. Motivation	439
73.2. Theory	439
73.2.1. The Chicken game	439
73.2.2. Stag Hunt	440
73.2.3. Schelling focal points	440
73.2.4. Arms race as iterated Prisoner's Dilemma	440
73.3. Implementation in R	440
73.3.1. Stag Hunt dynamics under varying risk dominance	440
73.3.2. Phase diagram: cooperation vs. defection basins	441
73.3.3. Arms race simulation	443
73.4. Worked example	444
73.4.1. Cuban Missile Crisis as a Chicken game	444
73.5. Extensions	446
Exercises	446
74. Empirical Case Studies	449
75. Empirical Case Studies	451
Learning objectives	451
75.1. Motivation	451
75.2. Theory	451
75.2.1. Spectrum auctions	451
75.2.2. Kidney exchange	452
75.2.3. Braess's paradox	452
75.3. Implementation in R	452
75.3.1. Spectrum auction simulation	452
75.3.2. Kidney exchange matching	453
75.3.3. Kidney exchange network visualization	454
75.3.4. Braess's paradox	456
75.3.5. Braess's paradox visualization	458
75.4. Worked example	460
75.4.1. Braess's paradox with a 4-node network	460
75.5. Extensions	461

Exercises	461
VI. Part VI — Ethics and the Future	463
76. Ethical Frameworks for Strategic AI	465
77. Ethical Frameworks for Strategic AI	467
Learning objectives	467
77.1. Motivation	467
77.2. Theory	467
77.2.1. Three ethical lenses	467
77.2.2. Social welfare functions	468
77.2.3. Impossibility and trade-offs	468
77.3. Implementation in R	468
77.3.1. Grid search over allocations	469
77.3.2. Pareto frontier with welfare optima	470
77.3.3. Welfare comparison across methods	471
77.4. Worked example	472
77.5. Extensions	473
Exercises	474
78. Fairness in Machine Learning	475
79. Fairness in Machine Learning	477
Learning objectives	477
79.1. Motivation	477
79.2. Theory	477
79.2.1. Fairness definitions	477
79.2.2. The impossibility theorem	478
79.2.3. Fairness as a multi-stakeholder game	478
79.3. Implementation in R	478
79.3.1. Simulating loan data	478
79.3.2. Unconstrained logistic regression	479
79.3.3. Fairness-penalized logistic regression	479
79.3.4. ROC curves by group	481
79.3.5. Accuracy-fairness trade-off frontier	482
79.4. Worked example	483
79.5. Extensions	485
Exercises	485
80. AI Alignment as a Game	487
81. AI Alignment as a Game	489
Learning objectives	489
81.1. Motivation	489
81.2. Theory	489
81.2.1. Principal-agent framework	489
81.2.2. Signaling game	489
81.2.3. Corrigibility in repeated games	490
81.3. Implementation in R	490
81.3.1. Signaling game payoffs	490
81.3.2. Equilibrium analysis	491

Table of contents

81.3.3. Signaling game tree visualization	491
81.3.4. Trust dynamics with Bayesian updating	493
81.4. Worked example	495
81.5. Extensions	496
Exercises	497
82. Cooperative AI	499
83. Cooperative AI	501
Learning objectives	501
83.1. Motivation	501
83.2. Theory	501
83.2.1. Commitment devices	501
83.2.2. Program equilibrium	501
83.2.3. Credible threats and commitment power	502
83.3. Implementation in R	502
83.3.1. Game definitions	502
83.3.2. Simulating commitment vs no commitment	503
83.3.3. Cooperation rates visualization	504
83.3.4. Welfare improvement via Pareto frontier	505
83.4. Worked example	507
83.5. Extensions	510
Exercises	510
84. The Future of Strategy	513
85. The Future of Strategy	515
Learning objectives	515
85.1. Motivation	515
85.2. Theory	515
85.2.1. Computational complexity of game solving	515
85.2.2. Emerging frontiers	516
85.2.3. Open problems in MARL	516
85.2.4. Responsible deployment	516
85.3. Implementation in R	516
85.3.1. Complexity landscape of game-theory problems	516
85.3.2. Timeline of game theory and AI convergence	518
85.4. Worked example	520
85.5. Extensions	523
Exercises	524
Appendices	525
A. R Refresher	525
B. R Refresher	527
B.1. Data structures	527
B.1.1. Vectors	527
B.1.2. Matrices	527
B.1.3. Lists	528
B.1.4. Data frames and tibbles	529

B.2.	Control flow	529
B.2.1.	Conditionals	529
B.2.2.	Loops	530
B.2.3.	The apply family	530
B.3.	Functions	531
B.3.1.	Defining functions	531
B.3.2.	Closures and environments	532
B.3.3.	Anonymous functions	532
B.4.	Tidyverse basics	533
B.4.1.	The pipe operator	533
B.4.2.	dplyr verbs	533
B.4.3.	Pivoting	534
B.4.4.	ggplot2 layers	535
B.5.	Useful idioms for this book	536
C.	Linear Algebra Essentials	539
D.	Linear Algebra Essentials	541
D.1.	Vectors and vector operations	541
D.2.	Matrices	542
D.2.1.	Matrix arithmetic in R	542
D.2.2.	Key matrix operations	542
D.3.	Systems of linear equations	543
D.3.1.	Solving $A\mathbf{x} = \mathbf{b}$	543
D.3.2.	Application: mixed Nash equilibrium	544
D.4.	Eigenvalues and eigenvectors	544
D.4.1.	Stability and the Jacobian	545
D.5.	Positive definite matrices	545
D.6.	Convex sets and the simplex	546
D.6.1.	The probability simplex	546
D.6.2.	Convex sets and convex combinations	546
D.6.3.	The support of a mixed strategy	547
D.7.	Matrix decompositions	547
D.7.1.	LU decomposition	547
D.7.2.	Singular value decomposition (SVD)	547
D.8.	Summary of R linear algebra functions	548
E.	Probability Essentials	549
F.	Probability Essentials	551
F.1.	Sample spaces and events	551
F.2.	Discrete distributions	551
F.2.1.	Common discrete distributions	551
F.2.2.	Probability mass function and CDF	552
F.3.	Continuous distributions	552
F.3.1.	Common continuous distributions	552
F.3.2.	Density, CDF, and quantiles	553
F.4.	Expected value and variance	553
F.4.1.	Linearity of expectation	554
F.5.	Conditional probability	554
F.6.	Bayes' theorem	555
F.6.1.	Sequential updating	555

Table of contents

F.7. Law of large numbers and Monte Carlo	556
F.7.1. Monte Carlo estimation	557
F.8. Random sampling for games	558
F.9. Summary of R probability functions	558
G. Exercise Solutions	559
H. Exercise Solutions	561
Chapter 1: What is a Game?	561
Chapter 2: Normal-Form Games	561
Chapter 3: Nash Equilibrium	562
Chapter 4: Mixed Strategies	563
Chapter 5: Extensive-Form Games	563
Chapter 6: Bayesian Games	564
Chapter 7: Repeated Games	565
Chapter 8: Cooperative Game Theory	565
Additional selected solutions	566
I. Glossary	569
J. Glossary	571

1. Preface

Preface

This book exists because two powerful ideas — game theory and computation — deserve to meet in one place, with working code you can run, modify, and extend.

1.1. Who this book is for

Strategy in R is written for graduate students, researchers, and practitioners who already have working fluency in R and want to apply it to strategic interaction. You should be comfortable with `tidyverse` idioms, writing functions, and reading mathematical notation. No prior game theory background is assumed; Part I builds the foundations from scratch.

If you are primarily a game theorist curious about computational methods, start with Part I and read linearly. If you are an R programmer looking for new domains, feel free to dip into Part III (simulation) or Part IV (AI/ML) and refer back to the theory chapters as needed.

1.2. What this book covers

The book is organized into six parts:

- **Part I — Foundations of Game Theory.** The core concepts: normal-form and extensive-form games, Nash equilibrium, mixed strategies, Bayesian games, repeated games, and cooperative game theory.
- **Part II — The R Toolkit.** A practical tour of R packages for game-theoretic computation, including `GameTheory`, `CoopGame`, `gtree`, and Python interop via `reticulate`.
- **Part III — Simulation.** Monte Carlo methods, agent-based models, Axelrod’s tournament, spatial games, replicator dynamics, evolutionary stability, network games, and performance optimization.
- **Part IV — AI and Machine Learning.** Reinforcement learning, Q-learning in games, multi-agent RL, self-play, counterfactual regret minimization, GANs as minimax games, and LLM agents.
- **Part V — Applications.** Auctions, mechanism design, matching markets, bargaining, and empirical case studies.
- **Part VI — Ethics and the Future.** Ethical frameworks, fairness in ML, AI alignment, cooperative AI, and open problems.

1.3. What this book does not cover

This is not a pure mathematics textbook — we state theorems but emphasize computation and intuition over formal proofs. It is not a machine learning textbook — we cover ML only where it intersects with strategic interaction. And it is not a software engineering manual — while we care about reproducible code, we do not cover R package development or deployment.

1.4. How to run the code

All code in this book is designed to run reproducibly. The repository uses `renv` for R package management. To get started:

```
git clone https://github.com/r-heller/strategy-in-r.git
cd strategy-in-r
```

Then in R:

```
renv::restore()
```

A handful of chapters use Python via `reticulate` (CFR, deep RL). These require a Python environment with packages listed in `python/requirements.txt`.

To render the full book:

```
quarto render
```

1.5. Conventions

- All R code uses `tidyverse` style and is designed to run end-to-end when the chapter is rendered.
- Figures use a colorblind-safe palette (Okabe-Ito) and are saved as both PDF and PNG at 300 dpi.
- Cross-references follow Quarto conventions: `@sec-...` for sections, `@fig-...` for figures, `@tbl-...` for tables, `@eq-...` for equations.
- Each chapter ends with exercises; solutions appear in Appendix H.
- Citations follow APA style. Every reference has a verified DOI, arXiv ID, or PubMed ID.

1.6. Acknowledgments

This book was written with the assistance of Claude (Anthropic) for drafting, code generation, and editorial review. All content has been verified, tested, and curated by the author. Errors remain my own.

2. Introduction: Why Strategy in R?

Why study strategic interaction computationally, and why R is the right tool for the job.

3. Introduction

3.1. The computational turn in game theory

Game theory began as a branch of mathematics. Von Neumann and Morgenstern (1944) laid the foundations; Nash (1950) proved that every finite game has an equilibrium. For decades, the field advanced primarily through theorems and proofs.

But the questions that matter most — *Which equilibrium will players actually reach? How do strategies evolve in populations? Can machines learn to play well?* — are fundamentally computational questions. They require simulation, numerical methods, and algorithm design. This book is about answering those questions with working code.

3.2. Why R?

R is the lingua franca of statistical computing and data science. It has mature ecosystems for visualization (`ggplot2`), data manipulation (`dplyr`), differential equations (`deSolve`), network analysis (`igraph`), and interactive graphics (`plotly`). For researchers who already think in R, there is no reason to context-switch to another language for game-theoretic computation.

Where R falls short — deep learning, counterfactual regret minimization — we use Python via `reticulate`, keeping the analysis pipeline in R while calling specialized Python libraries.

3.3. The structure of this book

The six parts of this book form a progression:

1. **Foundations** build the conceptual vocabulary: what games are, how to solve them, what equilibrium means.
2. **The R Toolkit** introduces the packages and utilities we use throughout.
3. **Simulation** moves from analytical solutions to computational experiments: Monte Carlo, agent-based models, evolutionary dynamics.
4. **AI and Machine Learning** explores how learning agents interact strategically: reinforcement learning, self-play, and frontier methods.
5. **Applications** grounds the theory in real domains: auctions, matching markets, bargaining, and empirical case studies.
6. **Ethics and the Future** asks what happens when strategic AI systems interact with humans and each other.

Each chapter is self-contained enough to read independently, but the parts are designed to be read in order for a coherent narrative.

3.4. How to use this book

Linear reading is best for graduate courses or self-study in game theory with a computational focus. Start at Part I, Chapter 1.

Reference reading works well for practitioners who need a specific tool. Jump to the relevant chapter, follow cross-references backward for prerequisites.

Project-based reading pairs well with the exercises. Each chapter ends with problems that can serve as starting points for course projects or research extensions.

Let's begin.

Part I.

Part I — Foundations of Game Theory

